

Transformers and reinforcement learning for the team orienteering problem under dynamic conditions

Guerrero, Escoto, Ammouriova, Men and Juan — Universitat Politècnica de València

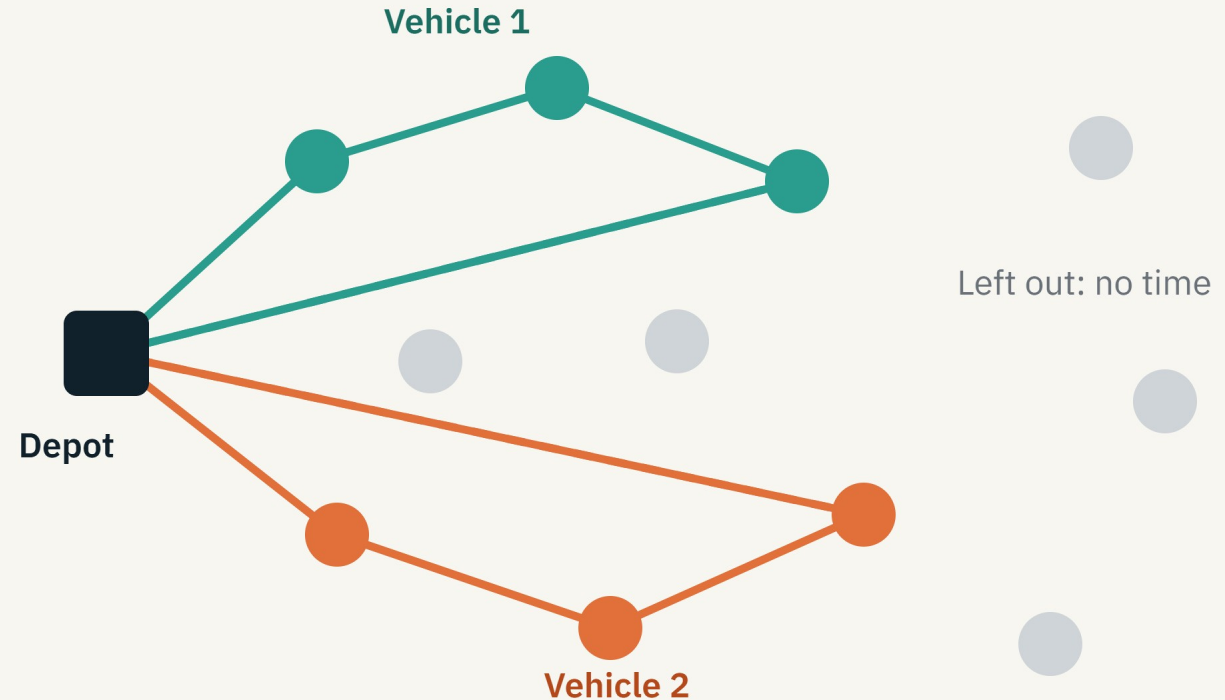
The team orienteering problem

Every stop carries a reward.

All vehicles leave from one depot and must reach the end depot.

Each vehicle has a time budget, so some stops stay unvisited.

Maximise reward, not coverage.



NP-hard: the number of possible route sets explodes with the number of stops

Travel time is not a constant

$$f(i, j) = d(i, j) \times (1 + 0.0625 \cdot \text{traffic} + 0.0625 \cdot \text{weather})$$

$d(i, j)$

The plain distance on the map.

traffic

Congestion, from 0 (clear) to 1 (jammed).

weather

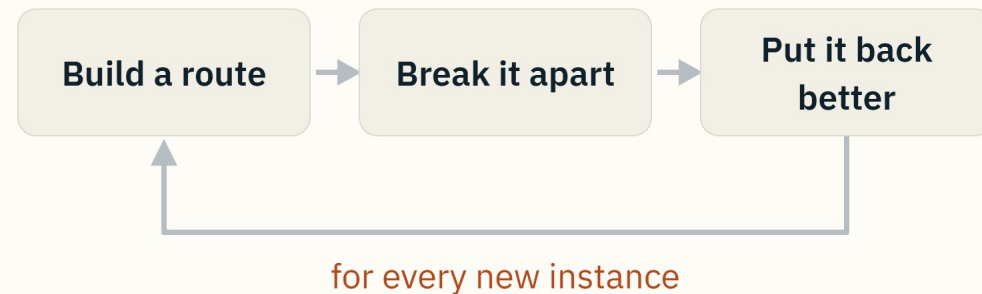
Rain, wind or snow, also 0 to 1.

Worst case, a leg takes 12.5% longer than the plan assumed.

A plan that fits exactly in the budget is therefore a plan that will break

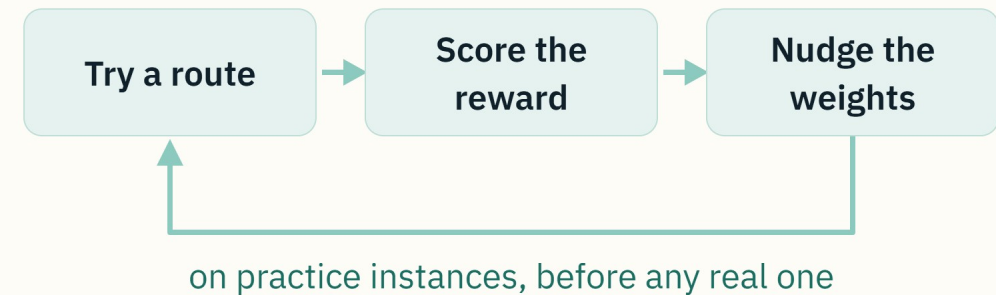
Both methods loop. They loop at different moments.

Search — the VNS baseline



The loop runs while the courier waits: about **4.5 seconds** before a plan exists.

Learning — the RL model



The loop runs once, offline: **40 hours**. Afterwards a plan takes 0.34 s.

Read the map once, then decide stop by stop

ONCE
per instance

The instance: stops, rewards,
vehicle budgets

Embed: each stop and vehicle
becomes a vector

Encode: attention relates
every stop to the rest

EVERY
step of the route

Where am I, and how much
time is left?

Which stops can I reach and
still get home?

Pick one of them, and move

until every vehicle is back at the end depot

The encoding is computed once. Only the context and the list of allowed stops change at each step.

The idea that makes it work

A STOP IS OFFERED ONLY IF

cost to reach it + cost to get home from it $\leq$ time left

The way there is predicted

A small network estimates what traffic and weather will cost on that exact segment.

The way home is guessed high

Conditions later on cannot be known, so the distance is inflated by a learned worst-case factor.

One loss trains both: REINFORCE for the routing, mean squared error for the prediction, joined by a weight λ .

Feasibility is enforced by construction, not repaired afterwards

What the numbers say

WHEN TRAVEL TIMES ARE FIXED

A tie on quality, 13× on speed

Average reward, higher is better

RL model  **11.84**

VNS  **11.40**

Seconds to produce one plan, lower is better

RL model  **0.34 s**

VNS  **4.55 s**

WHEN TRAVEL TIMES CHANGE

Only one of them finishes on time

Average reward, same architecture, same training

Predicts the delays  **11.26**

Ignores the delays  **2.54**

2.54 is not bad routing. Those vehicles arrive late, the route is thrown out, and it scores nothing.

The dynamic model broke the limit zero times.

TAKEAWAY

Modelling the uncertainty mattered more than solving the problem better.

WORTH ASKING ABOUT

Training and test instances come from the same uniform distribution.

Around 40 hours of training for each dynamic model.

No re-optimisation once a vehicle is already on the road.

Thank you — questions?